

JavaScript øving 1

Det er ingen innlevering av denne øvingen.

Lab tid for denne øvingen er uke 39 (21 til 25. september).

Oppgave 1: IntelliJ IDEA for statiske websider

JavaScript i DAT108 trenger et program for å skrive ren tekst.

JavaScript delen av DAT108 trenger ingen web-tjener, og HTML- og JavaScript-dokumentene kan nettleser aksessere som filer på datamaskinen. Det går fint å bruke en teksteditor som f.eks. Notepad, Vi og Emacs. Observer at ord-prosesserings program som f.eks. Word, Openoffice og Libreoffice ikke egner seg.

En IDE gir mer hjelp til å skrive koden, og her i denne oppgaven skal vi bruke IntelliJ IDEA for å lage en enkel webside med litt JavaScript kode. IntelliJ IDEA har innebygget webtjener som brukes for å vise webdokument.

IntelliJ IDEA har flere prosjekttyper for webutvikling. For et statisk webprosjekt kan dere bruke f.eks. *Empty Project* og *HTML*. Se side 2 og 3 i JavaScript forelesningen *Introduksjon* og opprett et webprosjekt *JavaScriptIntro*.

Det vedlagte arkivet (js1.zip og js1.tar.xz) inkludere koden fra oppgave 1 og oppgave 6. Pakk ut arkivet og kopier eller flytt innholdet til mappen for prosjektet som ble opprettet av IntelliJ. Høyreklikk så på prosjektet i IntelliJ og velg *Reload from Disk*.

Du kan velge å bruke en annen IDE, men da må du klare selv å konfigurere den.

Lag et HTML5 dokument med IntelliJ med følgende innhold:

```
<!DOCTYPE html>
<html lang="no">

  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <meta name="Author" content="Ditt navn">
    <link rel="shortcut icon" href="#">
    <script defer src="tallsystem.js"></script>
    <style>
      .hidden {
        display: none;
      }
    </style>
  </head>
</html>
```

```

    </style>

    <title>Tallsystem</title>
</head>

<body>
    <h1>Tallsystem</h1>

    <fieldset id="root">
        <legend>Tallsystem</legend>
        <input type="number" placeholder="Sett inn heltall">
        <input type="number" placeholder="Tallsystem">
        <button type="button">Finn svar</button>
        <p class="hidden">Tallet <span></span> i <span></span>-tallsystem
            er <span></span>.</p>
    </fieldset>
</body>
</html>

```

I samme mappe lager du et dokument *tallsystem.js* med følgende innhold:

```

const root = document.getElementById("root");

root.getElementsByTagName("button")[0].addEventListener("click", regnut);

function regnut() {
    const tallElement = root.getElementsByTagName("input")[0];
    const radixElement = root.getElementsByTagName("input")[1];
    const tall = parseInt(tallElement.value);
    const radix = parseInt(radixElement.value);

    const resultatElement = root.getElementsByTagName("p")[0];

    if ((tall > 0) && (radix > 2) && (radix <= 36)) {
        resultatElement.getElementsByTagName("span")[0].textContent = tall;
        resultatElement.getElementsByTagName("span")[1].textContent = radix;
        resultatElement.getElementsByTagName("span")[2].textContent =
            tall.toString(radix);
        resultatElement.classList.remove("hidden");
    } else {
        resultatElement.classList.add("hidden");
    }
}

```

Vis websiden i nettleser. Fyll så inn heltall i input elementene og klikk på knappen i websiden. Forsøk så å forklare til en annen student hva som skjer, og hvorfor.

Sjekk ut websiden til MDN for bruken av:

- [getElementById](#),
- [getElementsByTagName](#),
- [addEventListener](#),

- Egenskap [value](#) til et input element,
- HTML-node sin egenskap [textContent](#),
- **Number** sin metode [toString](#),
- HTML-element sin egenskap [classList](#).

Oppgave 2

Lag en webside med et INPUT element for å lese en temperatur i Fahrenheit. Ved klikk på en knapp skal websiden vise temperaturen i Celsius.

Oppgave 3

Ta utgangspunkt i oppgave 2, og la bruker velge med et SELECT element om temperaturen er oppgitt i Celsius eller Fahrenheit. Ved klikk på knappen skal websiden vise temperaturen i motsatt måleenhet.

Oppgave 4

Lag en kalkulator som utfører de fire grunnleggende aritmetiske operasjonene (addisjon, subtraksjon, multiplikasjon, og divisjon) på tall gitt fra brukeren. Brukeren skal først bli spurt om hvor mange tall som det ønskes operasjoner på, hvorpå det skal bli generert et tilsvarende antall input-felt og knapper med operasjoner.

Oppgave 5

Lag en websiden med to HTML-knapper (dvs. tagg **BUTTON**), push og pop, og et input felt (dvs. tagg **INPUT**) der bruker kan skrive noe tekst.

Når bruker klikker på push-knappen skal teksten pushes på en stabel, og input elementet tømmes.

Ved klikk på pop-knappen skal verdien på toppen av stabelen fjernes fra stabelen og vises til bruker, f.eks. inne i et **SPAN** element.

Tips: Du kan løse denne oppgaven med **Array**. Sjekk bruken av Array-metodene [push](#) og [pop](#).

Oppgave 6

Lag et lite webdokument med følgende innhold:

```
<!DOCTYPE html>
<html lang="no">

<head>
  <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
  <meta name="Author" content="Ditt navn" />
  <link rel="shortcut icon" href="#">
  <script defer src="desimaltall.js"></script>
  <style>
    .hidden {
```

```

        display: none;
    }
</style>

<title>Desimaltall</title>
</head>

<body>
    <h1>Desimaltall</h1>

    <fieldset id="root">
        <legend>Desimaltall</legend>
        <input type="number" placeholder="Sett inn desimaltall" step="any">
        <button type="button">Vis tall</button>
        <p class="hidden">Tallet er <span></span>.</p>
    </fieldset>
</body>
</html>

```

Lag et dokument *desimaltall.js* i samme mappe med følgende innhold:

```

const root = document.getElementById("root");

root.getElementsByTagName("button")[0].addEventListener("click", vistall);

function vistall() {
    const tallElement = root.getElementsByTagName("input")[0];
    const tallSomStreng = tallElement.value;

    resultatElement = root.getElementsByTagName("p")[0];
    if (tallSomStreng !== "") {
        resultatElement.getElementsByTagName("span")[0].textContent =
            tallSomStreng;
        resultatElement.classList.remove("hidden");
    } else {
        resultatElement.classList.add("hidden");
    }
}

```

Forsøk nå å fylle inn et desimaltall med desimalkomma, f.eks. «23,456». Klikk på knappen. Hva blir resultatet i følgende nettlesere?

- Microsoft Edge,
- Firefox,
- Opera,
- Google Chrome?

Hva blir resultatet for desimaltall med punktum som desimalskille?

I Firefox, oppfriske siden, eller slett cachen. Hva skjer da med tall med desimalpunktum i input-elementet? Forsøk å endre språkoppsett for nettleser, til f.eks. «en_US». Hva blir resultatet da?

Hvordan skiller nettbankene innlesing av kroner og øre?

Oppgave 7

Modifiser koden i oppgave 6 til å forlange at desimaltallet bruker desimalkomma. Et HTML input-element for desimalkomma kan være:

```
<input type="text" placeholder="Sett inn desimaltall" pattern="\d*,?\d*"
      title="Bruk kun sifre og desimalkomma">
```

Oppgave 8

Lag en webside med et [HTML TEXTAREA](#) element der bruker kan fylle inn en mengde med heltall og desimaltall med komma som desimalskille.

Lag et JavaScript program for å summere tallene, og som så viser resultatet på websiden. Input som ikke kan konverteres til et tall eller desimaltall skal ignoreres i utregningen. Programmet skal på websiden vise alt input som ble ignorert og som ikke er med i utregningen av summen.

Oppgave 9

Bruk dokumentasjon som du finner på nettet til å forklare forskjellen på *for...in* og *for...of* konstruksjonene.

Lag et JavaScript-program som demonstrerer forskjellen, og lag et webdokument for å kjøre programmet.

Oppgave 10

Lag en webside som leser inn i en **Array** en list av positive heltall fra et fra et [HTML TEXTAREA](#) element.

Bruk så **Array** sin metode [filter](#) til å filtrere bort alle tall som ikke er delelig med 3, og så [forEach](#) til å lage en tekststreng av alle tallene delelig med 3. Tekststrengen med tallene skal så skal vises f.eks. i et [HTML SPAN](#) element.

Oppgave 11

Lag en webside som leser inn i en **Array** en list av positive heltall fra et fra et [HTML TEXTAREA](#) element.

Legg også inn på websiden et [HTML SELECT](#) element der bruker kan velge mellom *Sum*, *Produkt*, *Maks* eller *Min*. Bruk **Array** sin metode [reduce](#) til å finne regne ut verdien.

Oppgave 12

Lag en webside der bruker kan oppgi to datoer ved å bruke to HTML INPUT element:

```
<input type="date" required>
```

Lag så et JavaScript program som viser antall dager mellom de to datoene, f.eks. i et [HTML SPAN](#) element.

Oppgave 13

Lag en webside der bruker kan oppgi en dato og en land-språkkode. Eksempel på en land-språkkode er f.eks. «nb-NO» for norsk bokmål. Websiden skal vise datoen sin ukedag (f.eks. Mandag) ved å bruke den oppgitte land-språkkoden. Ukedagen kan vises f.eks. i et [HTML SPAN](#) element.

Tips: Sjekk f.eks. **Date** objektet sin metode [toLocaleDateString](#).

For land-språkkoder, se f.eks. [International Components for Unicode \(ICU\) Data](#), [Country Code Language List](#), eller www.iana.org.

Oppgave 14

Lag en web-applikasjon med JavaScript for å trille en terning. Funksjonaliteten til terningen skal implementeres som en JavaScript klasse **Terning**. Terningen må knyttes til HTML elementer for å trille terningen og vise resultatet. I denne oppgaven skal du bruke en JavaScript klasse **TerningKontroll** til dette.

Websiden for å trille terning kan inneholde følgende HTML:

```
<fieldset id="root">
  <legend>Trill terning</legend>
  <button type="button">Trill terning</button>
  <p class="hidden">Resultat: <span></span></p>
</fieldset>
```

Klassen **TerningKontroll** knytter terningen til HTML koden over, og kan kalles på følgende måte:

```
const rootElement = new DiceController("root");
new DiceController(rootElement);
```

I koden over er *root* verdien til ID-attributtet til [HTML FIELDSET-elementet](#) for terningen. For at løsningen skal være portabel må **TerningKontroll** referer HTML elementer relativt til FIELDSET-elementet. JavaScript koden skal altså ikke bruke HTML ID-attributter for å referer elementer inne i FIELDSET-elementet.

Oppgave 15 – FORM validering

Første del av denne oppgaven skal løses kun ved bruk av HTML og CSS, dvs. uten bruk av JavaScript. I del to må JavaScript brukes for å sjekke om input-elementet sin tilstand er gyldig.

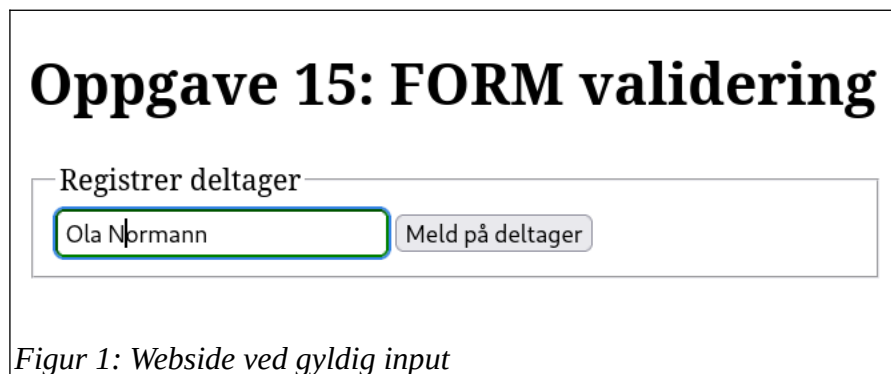
Del en: HTML og CSS

En webside har et input-felt for å registrere en navn på deltager i et arrangement. Hvis navnet er ugyldig skal rammen på input-elementet vises med rød farge.

Et gyldig navn kan bestå av flere delnavn skilt av mellomrom. Lovlige tegn er kun bokstaver og tegnet «-», altså bindestrek.

Hvert delnavn må starte med en stor bokstav etterfulgt av kun små bokstaver. Bindestrek kan kun benyttes mellom delnavn. Det skal ikke være mellomrom før eller etter bindestrek.

Skjermbildet under viser hvordan websiden kan se ut ved gyldig input:



Figur 1: Webside ved gyldig input

Neste figur viser hvordan websiden kan se ut ved ugyldig input:



Figur 2: Webside ved ugyldig input

En forklarende tekst skal vises når musen er over INPUT-elementet.

Oppførselen over er implementert kun ved bruk av HTML og CSS, uten bruk av JavaScript.

Del to: JavaScript

Ved klikk på knappen *Meld på deltager* skal websiden vise en tekst som angir om input er gyldig eller ugyldig, slik figuren under viser:



Figur 3: Klikk på knappen skal vise om input er gyldig eller ugyldig

Oppførsel ved klikk på knappen må løses ved å skrive JavaScript kode.

Nyttige lenker

Følgende lenker kan være nyttig for å løse denne oppgaven:

- HTML og CSS:
 - CSS egenskapen [border-color](#)
 - CSS pseudoklassene [valid](#) og [invalid](#)
 - HTML INPUT elementet sitt attributt [pattern](#)
- JavaScript:
 - [HTMLInputElement: validity property](#)
 - [ValidityState: valid property](#)
- Regulære uttrykk:
 - [Regulære uttrykk](#)
 - [Tegnklasser](#) i regulære uttrykk
 - [Unicode escape](#) for tegnklasse
 - RegExp eksempler fra forelesningene om ECMA:
 - [RegExp](#)
 - [Delstrenger, søk i tekst og erstatte tekst](#) : demonstrasjonene demo3 til demo7

Oppgave 16. Konsistens mellom FORM elementer

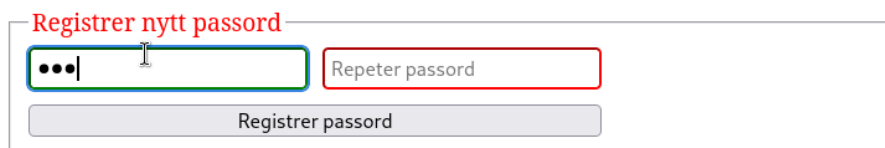
Når konsistens mellom felter skal kontrolleres må JavaScript brukes. Ved inkonsistens kan JavaScript endre et element sin tilstand til å være ugyldig. Når et element er ugyldig vil CSS pseudotilstand *invalid* kunne markere elementet som ugyldig på websiden.

Lag et webside med to INPUT elementer, ett for å fylle inn et passord og ett for å repetere passordet. Når tekst fylles inn i et av feltene (hendelse *input*) skal JavaScript kode sjekke om begge input har samme verdi. Hvis verdiene er ulike settes tilstand for feltet for repetert passord til ugyldig.

Et INPUT felt sin tilstand kan settes til ugyldig ved å registrere en ikke-tom feilmeldingstekst med metoden [setCustomValidity](#). Tilstanden settes tilbake til gyldig ved å registrere en tom tekst med [setCustomValidity](#).

I figuren under har bruker fylt inn et passord i passordfeltet. Da er passord og repetert passord ulike. CSS regel for pseudotilstand *invalid* skal da markere repetert passord med rød ramme.

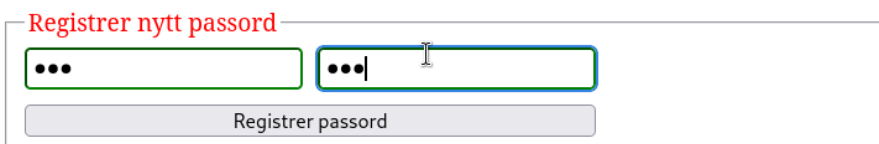
Oppgave 16: Validering av konsistens mellom INPUT



Figur 4: Ugyldig repetert passord

Når passord og repetert passord er like skal repetert passord ha tilstand valid, som vist i figuren under:

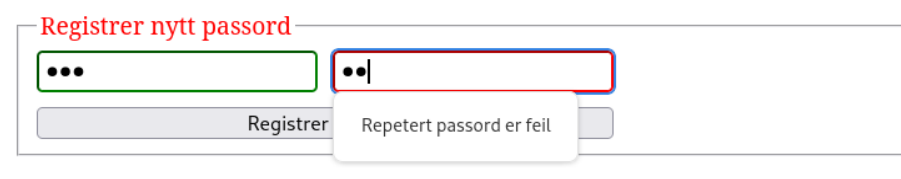
Oppgave 16: Validering av konsistens mellom INPUT



Figur 5: Gyldig repetert passord

Ved klikk på knappen *Registrer passord* skal en informasjonsboks vises hvis repetert passord er ugyldig:

Oppgave 16: Validering av konsistens mellom INPUT



Figur 6: Informasjonsboks ved ugyldig repetert passord

Informasjonsboksen vil vises for et ugyldig INPUT element ved å kalle [HTMLInputElement](#) sin metode [reportValidity](#).

Oppgave 17

En webside for en konkurranse skal ha et input-felt for navn på deltager, et input-felt for totaltid i minutter (heltall) og en knapp for å registrere deltager.

Bruk løsningen fra oppgave 15 til å validere input-felt for deltager uten bruk av JavaScript.

Når knappen for å registrere deltager blir klikket skal deltageren vises i et HTML TABLE element sortert med beste deltager øverst.

Husk at en uærlig bruker kan forsøke å ta kontroll over websiden ved å legge kode i INPUT-feltene. Verdiene herfra må legges inn i tabellen som ren tekst, f.eks. ved bruker av egenskapen [textContent](#).

Observer at HTML tabellen kun skal oppdateres med en ny rekke for den sist registrerte deltageren. Det er unødvendig å mellomlagre deltagerne i en JavaScript Array.

Bruk egenskaper og metoder til HTML TABLE, TR, TD, TH, TBODY og THEAD for å løse denne oppgaven:

- HTMLTableElement (HTML tagg TABLE) egenskaper og metoder:
 - egenskap [tBodies](#)
 - egenskap [tHead](#)
 - egenskap [rows](#)
 - metode [insertRow](#)
- HTMLTableSectionElement (HTML tagger THEAD og TBODY) egenskaper og metoder:
 - egenskap [rows](#)
 - metode [insertRow](#)
- HTMLTableRowElement (HTML tagg TR) egenskaper og metoder:
 - egenskap [cells](#)
 - metode [insertCell](#)

Oppgave 18

Løs oppgave 17 ved å bruke grunnleggende DOM metoder for å bygge tabellinnholdet opp fra grunnen:

- [createElement](#)
- [createTextNode](#)
- [append](#)
- [appendChild](#)
- [after](#)
- [before](#)
- [insertBefore](#)

Oppgave 19

Når webinnhold skal bygges med JavaScript er raskeste metode ofte å klonе en webstruktur som allerede finnes i nettleser sin hukommelse.

I oppgave 17 og 18 laget vi en ny tabellrekke for hver deltager. Da grunnstrukturen i tabellrekken er den samme for alle deltagere er en raskere løsning å klonе en eksisterende rekke. En løsning kan være å klonе en annen rekke i samme tabell for så å endre innholdet. En bedre løsning er å opprette en tabellrekke-mal i nettleser sin hukommelse og så klonе den for hver ny deltager.

Et [DocumentFragment](#) er et beholderelement for HTML-elementer i nettleser sin hukommelse. Dette elementet vises ikke i nettleser, men kan klones og så legges inn i web-strukturen med JavaScript.

Et DocumentFragment kan legges inn i HTML-koden ved å bruke HTML tagg [TEMPLATE](#), men kan også opprettes med JavaScript. JavaScript-koden nedenfor oppretter et DocumentFragment for en tabellrekke som kan benyttes for deltagere i konkurransen:

```
const trtemplate = document.createElement("template");
trtemplate.innerHTML = "<tr><td></td><td></td></tr>";
```

Da et HTML-element kun kan brukes en gang i en web-struktur må dokument-fragmentet kopieres (metode [cloneNode](#)). Kopien kan så legges inn i web-dokumentet. Koden nedenfor lager en kopi av dokument-fragmentet med tabellrekken:

```
const trkopi = trtemplate.content.cloneNode(true);
```

Nå vil *trkopi* inneholde en tabellrekke som kan legges inn i tabellen med DOM-metoder.

Løs oppgave 17 på nytt, men nå ved å bruke dokument-fragmentet *trtemplate*.